

TORQUEHUB

Sistema de Gestão para Oficinas Automotivas

Documento de Especificação de Requisitos de Software — SRS

Projeto: TorqueHub

Versão: 1.0

Status: Planejamento / Desenvolvimento

Tipo: Aplicação Web

Público inicial: Oficinas de pequeno e médio porte

Escopo inicial: Gestão de oficinas com foco em motocicletas

Expansão planejada: Automóveis e outros tipos de veículos

1. INTRODUÇÃO

1.1 Objetivo do documento

Este documento tem como objetivo definir os requisitos, regras de negócio, funcionalidades, restrições e características esperadas do sistema TorqueHub.

O documento será utilizado como referência para as etapas de análise, modelagem, desenvolvimento, testes e evolução do sistema.

A especificação separa os comportamentos que o sistema deverá oferecer das características de qualidade esperadas, permitindo que os requisitos sejam utilizados posteriormente como base para desenvolvimento e testes.

2. VISÃO GERAL DO SISTEMA

O **TorqueHub** será um sistema web destinado ao gerenciamento de oficinas automotivas.

A primeira versão será desenvolvida com foco em **oficinas de motocicletas**, permitindo o gerenciamento de clientes, veículos, funcionários, mecânicos, serviços, peças, estoque e ordens de serviço.

Apesar do foco inicial em motocicletas, o sistema será projetado desde o início para trabalhar com o conceito genérico de **veículo**.

Dessa forma, versões futuras poderão incorporar:

- Carros;
- Veículos utilitários;
- Vans;

- Caminhonetes;
- Outros tipos de veículos que façam sentido para a plataforma.

A expansão deverá ocorrer sem necessidade de reconstruir os principais módulos do sistema.

3. OBJETIVOS DO SISTEMA

O TorqueHub deverá:

1. Centralizar os dados da oficina;
 2. Organizar o cadastro de clientes;
 3. Registrar os veículos dos clientes;
 4. Controlar serviços oferecidos;
 5. Controlar peças e estoque;
 6. Registrar ordens de serviço;
 7. Acompanhar o andamento das manutenções;
 8. Registrar peças utilizadas;
 9. Registrar serviços realizados;
 10. Manter histórico de manutenção dos veículos;
 11. Auxiliar os funcionários na rotina da oficina;
 12. Fornecer informações resumidas através de um dashboard;
 13. Reduzir a dependência de controles manuais;
 14. Preservar o histórico das informações;
 15. Possibilitar futuras expansões para diferentes tipos de veículos;
 16. Possibilitar futuramente o gerenciamento de múltiplas oficinas.
-

4. ESCOPO DO PROJETO

4.1 Escopo da versão 1.0

A versão inicial será direcionada à gestão interna de uma oficina.

Os seguintes módulos farão parte da V1:

- Autenticação;
- Usuários;
- Clientes;
- Veículos;
- Mecânicos;
- Serviços;
- Peças;
- Estoque;
- Ordens de Serviço;
- Histórico de manutenção;
- Dashboard.

Na V1, o tipo de veículo inicialmente suportado será:

Motocicleta.

Entretanto, o modelo de dados utilizará o conceito de **Veículo**, evitando que as entidades principais sejam dependentes exclusivamente de motocicletas.

5. FUNCIONALIDADES FORA DO ESCOPO DA V1

As funcionalidades abaixo não serão obrigatórias na primeira versão:

- Cadastro de múltiplas oficinas;
- Marketplace;
- Busca pública de oficinas;
- Favoritar oficinas;
- Avaliações;
- Comentários;
- Agendamento online;
- Solicitação de orçamento online;
- Pagamento online;
- Aplicativo mobile;
- Notificações por WhatsApp;
- Rastreamento de localização;
- Integração com sistemas externos;
- Gerenciamento de carros.

Essas funcionalidades poderão ser implementadas em versões futuras.

6. ATORES DO SISTEMA

O TorqueHub possuirá inicialmente três perfis principais.

6.1 Administrador

Responsável pelo gerenciamento geral do sistema.

Poderá:

- Gerenciar usuários;
 - Gerenciar funcionários;
 - Gerenciar mecânicos;
 - Consultar informações gerais;
 - Visualizar indicadores;
 - Administrar configurações do sistema.
-

6.2 Funcionário

Responsável pelas atividades administrativas da oficina.

Poderá:

- Cadastrar clientes;
 - Cadastrar veículos;
 - Consultar clientes;
 - Consultar veículos;
 - Abrir ordens de serviço;
 - Consultar ordens;
 - Gerenciar serviços;
 - Gerenciar peças;
 - Consultar estoque;
 - Atualizar informações das ordens.
-

6.3 Mecânico

Responsável pela execução dos serviços.

Poderá:

- Visualizar ordens atribuídas;
 - Consultar dados do veículo;
 - Consultar o problema relatado;
 - Registrar diagnóstico;
 - Registrar observações técnicas;
 - Registrar serviços realizados;
 - Registrar peças utilizadas;
 - Atualizar o andamento da manutenção;
 - Finalizar serviços sob sua responsabilidade.
-

7. REQUISITOS FUNCIONAIS

RF01 — Autenticação

O sistema deverá permitir que usuários cadastrados realizem login utilizando e-mail e senha.

Após a autenticação, o sistema deverá identificar o perfil do usuário e disponibilizar as funcionalidades correspondentes às suas permissões.

RF02 — Logout

O sistema deverá permitir que o usuário autenticado encerre sua sessão.

Após o logout, o acesso às funcionalidades protegidas deverá ser bloqueado.

RF03 — Controle de acesso

O sistema deverá controlar o acesso às funcionalidades de acordo com o perfil do usuário.

Um usuário não deverá acessar funcionalidades para as quais não possua permissão.

RF04 — Cadastro de usuários

O administrador deverá poder cadastrar usuários.

O cadastro deverá possuir:

- Nome;
 - E-mail;
 - Senha;
 - Telefone;
 - Perfil;
 - Status.
-

RF05 — Gerenciamento de usuários

O administrador deverá poder:

- Consultar usuários;
- Editar usuários;
- Alterar perfil;
- Ativar usuário;
- Desativar usuário.

Usuários desativados não deverão conseguir realizar login.

8. MÓDULO DE CLIENTES

RF06 — Cadastro de clientes

O sistema deverá permitir cadastrar clientes.

O cadastro deverá possuir:

- Nome completo;
- CPF;
- Telefone;

- E-mail;
 - Endereço;
 - Observações;
 - Status.
-

RF07 — Consulta de clientes

O sistema deverá permitir pesquisar clientes por:

- Nome;
 - CPF;
 - Telefone.
-

RF08 — Atualização de clientes

Usuários autorizados deverão poder alterar os dados cadastrais de um cliente.

RF09 — Desativação de clientes

O sistema deverá permitir desativar clientes.

Clientes que possuam histórico de serviços não deverão ser excluídos fisicamente.

9. MÓDULO DE VEÍCULOS

RF10 — Cadastro de veículo

O sistema deverá permitir cadastrar veículos vinculados a clientes.

O cadastro deverá utilizar atributos comuns aos veículos, incluindo:

- Identificador;
 - Cliente proprietário;
 - Tipo;
 - Marca;
 - Modelo;
 - Ano;
 - Placa;
 - Cor;
 - Quilometragem;
 - Observações;
 - Status.
-

RF11 — Tipos de veículo

O sistema deverá possuir uma classificação para o tipo de veículo.

Na versão 1.0:

MOTOCICLETA

Futuramente poderão ser adicionados:

- CARRO;
- CAMINHONETE;
- VAN;
- UTILITÁRIO;
- OUTROS.

A inclusão de novos tipos não deverá exigir alteração na estrutura das ordens de serviço.

RF12 — Consulta de veículos

O sistema deverá permitir pesquisar veículos por:

- Placa;
 - Marca;
 - Modelo;
 - Cliente;
 - Tipo.
-

RF13 — Atualização de veículo

Usuários autorizados poderão atualizar os dados do veículo.

RF14 — Histórico do veículo

O sistema deverá permitir consultar o histórico de manutenção associado ao veículo.

O histórico deverá apresentar:

- Ordens de serviço;
- Datas;
- Serviços realizados;
- Peças utilizadas;
- Quilometragem;
- Valores;
- Diagnósticos;
- Observações;

- Status.
-

10. MÓDULO DE MECÂNICOS

RF15 — Cadastro de mecânicos

O sistema deverá permitir cadastrar mecânicos.

O cadastro deverá possuir:

- Nome;
 - CPF;
 - Telefone;
 - E-mail;
 - Especialidade;
 - Status.
-

RF16 — Consulta de mecânicos

O sistema deverá permitir pesquisar mecânicos por:

- Nome;
 - Especialidade;
 - Status.
-

RF17 — Atualização de mecânicos

Usuários autorizados poderão editar as informações dos mecânicos.

RF18 — Desativação de mecânico

O sistema deverá permitir desativar mecânicos.

Um mecânico desativado não poderá receber novas ordens de serviço, mas seu histórico deverá ser preservado.

11. MÓDULO DE SERVIÇOS

RF19 — Cadastro de serviços

O sistema deverá permitir cadastrar serviços oferecidos pela oficina.

Cada serviço deverá possuir:

- Nome;
 - Descrição;
 - Valor base;
 - Tempo estimado;
 - Status.
-

RF20 — Consulta de serviços

O sistema deverá permitir pesquisar serviços por nome e status.

RF21 — Atualização de serviços

Usuários autorizados poderão editar serviços cadastrados.

RF22 — Desativação de serviços

Serviços que não estejam mais disponíveis poderão ser desativados.

Serviços utilizados em ordens antigas deverão permanecer disponíveis no histórico.

12. MÓDULO DE PEÇAS E ESTOQUE

RF23 — Cadastro de peças

O sistema deverá permitir cadastrar peças.

Cada peça deverá possuir:

- Nome;
 - Código;
 - Fabricante;
 - Descrição;
 - Preço de custo;
 - Preço de venda;
 - Quantidade em estoque;
 - Estoque mínimo;
 - Status.
-

RF24 — Consulta de estoque

O sistema deverá permitir consultar:

- Peças disponíveis;
 - Quantidade atual;
 - Estoque mínimo;
 - Peças abaixo do estoque mínimo.
-

RF25 — Entrada de estoque

O sistema deverá permitir registrar entrada de peças no estoque.

A movimentação deverá registrar:

- Peça;
 - Quantidade;
 - Data;
 - Usuário responsável;
 - Observação.
-

RF26 — Saída de estoque

O sistema deverá registrar a saída de peças utilizadas em ordens de serviço.

RF27 — Histórico de estoque

O sistema deverá manter o histórico das movimentações de estoque.

13. MÓDULO DE ORDENS DE SERVIÇO

RF28 — Abertura de ordem de serviço

O sistema deverá permitir abrir uma nova ordem de serviço.

A ordem deverá estar vinculada a:

- Cliente;
 - Veículo;
 - Funcionário responsável;
 - Mecânico responsável.
-

RF29 — Dados da ordem de serviço

A ordem deverá possuir:

- Número;
 - Data de entrada;
 - Previsão de entrega;
 - Data de conclusão;
 - Quilometragem de entrada;
 - Problema relatado;
 - Diagnóstico;
 - Observações;
 - Status;
 - Valor total.
-

RF30 — Serviços da ordem

Uma ordem de serviço poderá possuir um ou vários serviços.

Para cada serviço deverão ser registrados:

- Serviço;
 - Quantidade;
 - Valor unitário;
 - Valor total.
-

RF31 — Peças da ordem

Uma ordem de serviço poderá possuir uma ou várias peças.

Para cada peça deverão ser registrados:

- Peça;
 - Quantidade;
 - Valor unitário;
 - Valor total.
-

RF32 — Cálculo do valor

O sistema deverá calcular o valor total da ordem de serviço.

A fórmula será:

Valor Total = Serviços + Peças

RF33 — Status da ordem

A ordem poderá possuir os seguintes estados:

1. Aberta;
 2. Em análise;
 3. Aguardando aprovação;
 4. Em manutenção;
 5. Aguardando peça;
 6. Finalizada;
 7. Entregue;
 8. Cancelada.
-

RF34 — Atualização de status

Usuários autorizados poderão atualizar o status da ordem conforme o andamento do serviço.

RF35 — Diagnóstico

O mecânico responsável poderá registrar o diagnóstico técnico do veículo.

RF36 — Observações técnicas

O mecânico poderá registrar observações relacionadas ao serviço.

RF37 — Finalização

Uma ordem poderá ser marcada como finalizada após a conclusão dos serviços.

O sistema deverá registrar a data de conclusão.

RF38 — Entrega

Uma ordem finalizada poderá ser marcada como entregue.

Ao realizar a entrega, o sistema deverá registrar a data de entrega.

RF39 — Cancelamento

Usuários autorizados poderão cancelar uma ordem de serviço.

Ordens canceladas deverão permanecer no histórico.

RF40 — Consulta de ordens

O sistema deverá permitir pesquisar ordens por:

- Número;
 - Cliente;
 - CPF;
 - Placa;
 - Marca;
 - Modelo;
 - Mecânico;
 - Status;
 - Data.
-

14. MÓDULO DE HISTÓRICO

RF41 — Histórico de manutenção

O sistema deverá manter o histórico das manutenções realizadas.

O histórico deverá estar associado ao veículo.

RF42 — Histórico por cliente

O sistema deverá permitir visualizar o histórico dos veículos pertencentes a um cliente.

RF43 — Histórico por veículo

O sistema deverá permitir visualizar todas as ordens de serviço relacionadas a determinado veículo.

15. DASHBOARD

RF44 — Dashboard administrativo

O sistema deverá disponibilizar um painel com indicadores gerais.

O dashboard deverá apresentar:

- Total de clientes;

- Total de veículos;
 - Total de mecânicos;
 - Ordens abertas;
 - Ordens em manutenção;
 - Ordens aguardando peça;
 - Ordens finalizadas;
 - Ordens entregues;
 - Peças com estoque baixo.
-

RF45 — Indicadores financeiros

O sistema poderá apresentar:

- Valor total de ordens;
- Valor de serviços;
- Valor de peças;
- Faturamento por período.

Esses indicadores deverão ser tratados como informações administrativas da oficina.

16. PESQUISA E FILTROS

RF46 — Pesquisa global

O sistema deverá permitir localizar registros utilizando campos de pesquisa.

RF47 — Filtros

As principais listagens deverão permitir filtros.

Exemplos:

- Status;
 - Período;
 - Cliente;
 - Mecânico;
 - Tipo de veículo.
-

17. REQUISITOS NÃO FUNCIONAIS

RNF01 — Usabilidade

A interface deverá ser simples, clara e intuitiva.

RNF02 — Responsividade

A interface deverá adaptar-se a diferentes tamanhos de tela, especialmente computadores e notebooks.

A arquitetura visual deverá permitir futura adaptação para dispositivos móveis.

RNF03 — Compatibilidade

O sistema deverá funcionar nos principais navegadores modernos:

- Google Chrome;
 - Microsoft Edge;
 - Mozilla Firefox.
-

RNF04 — Desempenho

As operações comuns deverão apresentar respostas rápidas em condições normais de utilização.

RNF05 — Segurança

O sistema deverá utilizar autenticação para proteger funcionalidades administrativas.

RNF06 — Senhas

As senhas não deverão ser armazenadas em texto puro.

Deverá ser utilizado mecanismo adequado de hash de senha.

RNF07 — Controle de acesso

O sistema deverá implementar autorização baseada no perfil do usuário.

RNF08 — Integridade

O banco deverá utilizar:

- Chaves primárias;
- Chaves estrangeiras;
- Restrições;

- Relacionamentos;
 - Validações.
-

RNF09 — Persistência

Os dados deverão ser armazenados em MySQL.

RNF10 — API

A comunicação entre o front-end e o back-end deverá ocorrer por meio de uma API REST desenvolvida em Python utilizando Flask.

RNF11 — Formato de comunicação

A API deverá utilizar JSON para troca de dados entre cliente e servidor.

RNF12 — Testabilidade

Os endpoints da API deverão ser testáveis individualmente utilizando Postman.

RNF13 — Manutenibilidade

O código deverá ser modularizado e organizado por responsabilidades.

RNF14 — Versionamento

O projeto deverá utilizar Git para controle de versão e GitHub para armazenamento dos repositórios.

RNF15 — Escalabilidade

A arquitetura deverá permitir a inclusão futura de novos tipos de veículos e novas funcionalidades sem necessidade de reconstrução completa do sistema.

RNF16 — Extensibilidade

A estrutura deverá permitir futuramente a inclusão de:

- Carros;

- Caminhonetes;
 - Vans;
 - Outros veículos;
 - Múltiplas oficinas;
 - Clientes externos;
 - Agendamento;
 - Avaliações.
-

18. REGRAS DE NEGÓCIO

RN01 — Proprietário do veículo

Todo veículo deverá estar vinculado a um cliente.

RN02 — Cliente

Um cliente poderá possuir um ou vários veículos.

RN03 — Veículo

Um veículo poderá possuir diversas ordens de serviço durante sua vida útil.

RN04 — Ordem de serviço

Cada ordem deverá estar associada a apenas um veículo.

RN05 — Serviços

Uma ordem poderá possuir vários serviços.

Um serviço poderá estar presente em várias ordens.

RN06 — Peças

Uma ordem poderá utilizar várias peças.

Uma peça poderá ser utilizada em várias ordens.

RN07 — Estoque

A quantidade de uma peça nunca poderá ficar abaixo de zero.

RN08 — Estoque insuficiente

O sistema não deverá permitir registrar utilização de uma quantidade superior ao estoque disponível.

RN09 — Histórico

Ordens concluídas, entregues ou canceladas deverão permanecer armazenadas para consulta histórica.

RN10 — Ordem entregue

Uma ordem somente poderá ser entregue após ser finalizada.

RN11 — Ordem cancelada

Uma ordem cancelada não poderá receber novos serviços ou peças.

RN12 — Mecânico inativo

Um mecânico inativo não poderá receber novas ordens.

Ordens antigas continuarão associadas a ele.

RN13 — Serviço inativo

Um serviço desativado não poderá ser adicionado a novas ordens, mas deverá permanecer no histórico das ordens antigas.

RN14 — Peça inativa

Uma peça desativada não poderá ser utilizada em novas ordens.

Seu histórico deverá ser preservado.

RN15 — Quilometragem

A quilometragem registrada em uma nova ordem não deverá ser inferior à última quilometragem conhecida do mesmo veículo.

RN16 — Exclusão de dados históricos

Registros que possuam dependências históricas não deverão ser excluídos fisicamente.

Quando necessário, deverá ser utilizada desativação lógica.

RN17 — Valor dos itens

O valor de cada item deverá ser registrado no momento em que for adicionado à ordem.

Isso permitirá preservar o valor histórico mesmo que o preço atual do serviço ou peça seja alterado posteriormente.

RN18 — Valor total

O valor total da ordem deverá ser composto pelos valores dos serviços e das peças registrados.

RN19 — Tipo de veículo

O tipo do veículo deverá ser armazenado de forma que novos tipos possam ser adicionados futuramente.

A estrutura da ordem de serviço não deverá depender exclusivamente de motocicletas.

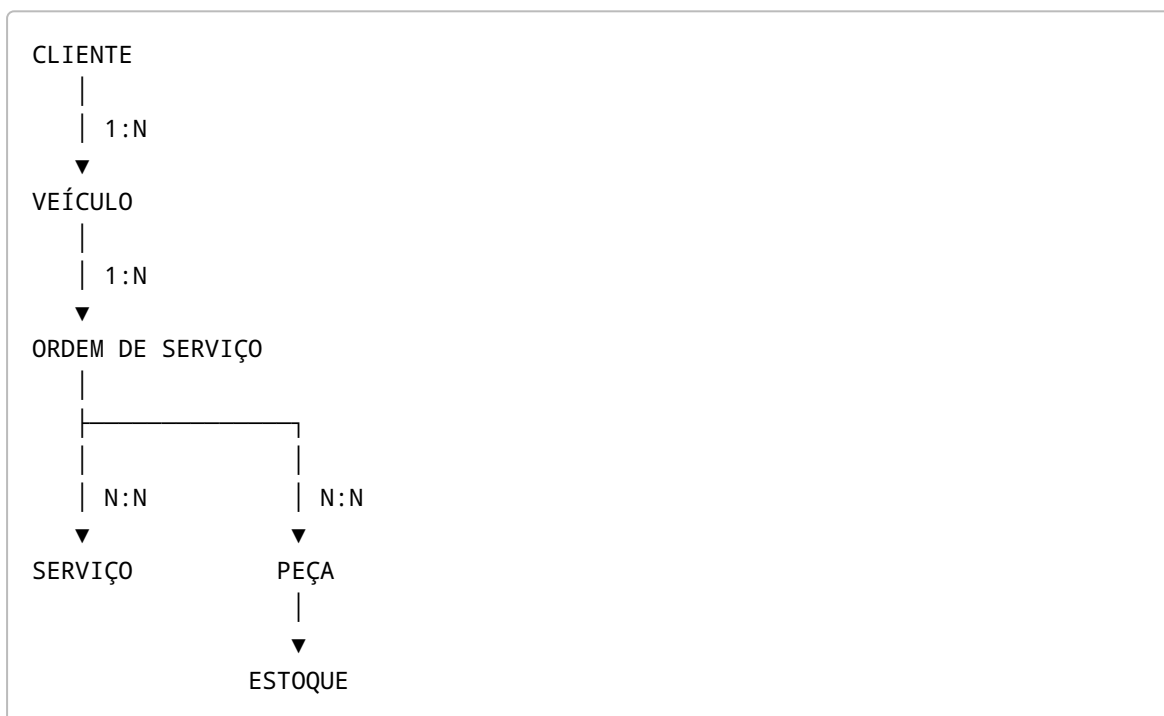
RN20 — Expansão

A inclusão de um novo tipo de veículo não deverá exigir alterações nas entidades principais de:

- Cliente;
 - Ordem de Serviço;
 - Serviço;
 - Peça;
 - Estoque;
 - Histórico.
-

19. MODELO CONCEITUAL INICIAL

A estrutura conceitual deverá seguir a seguinte lógica:

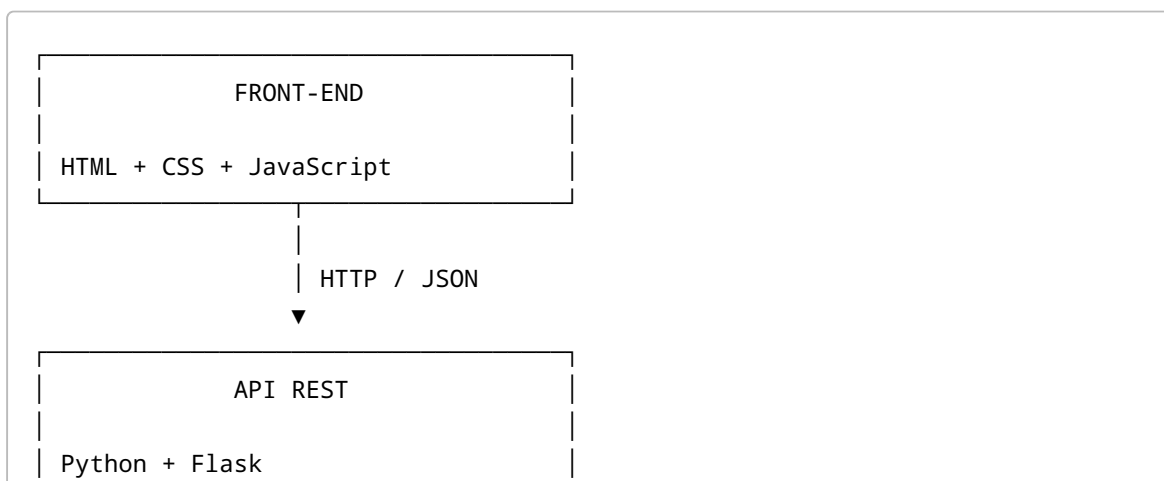


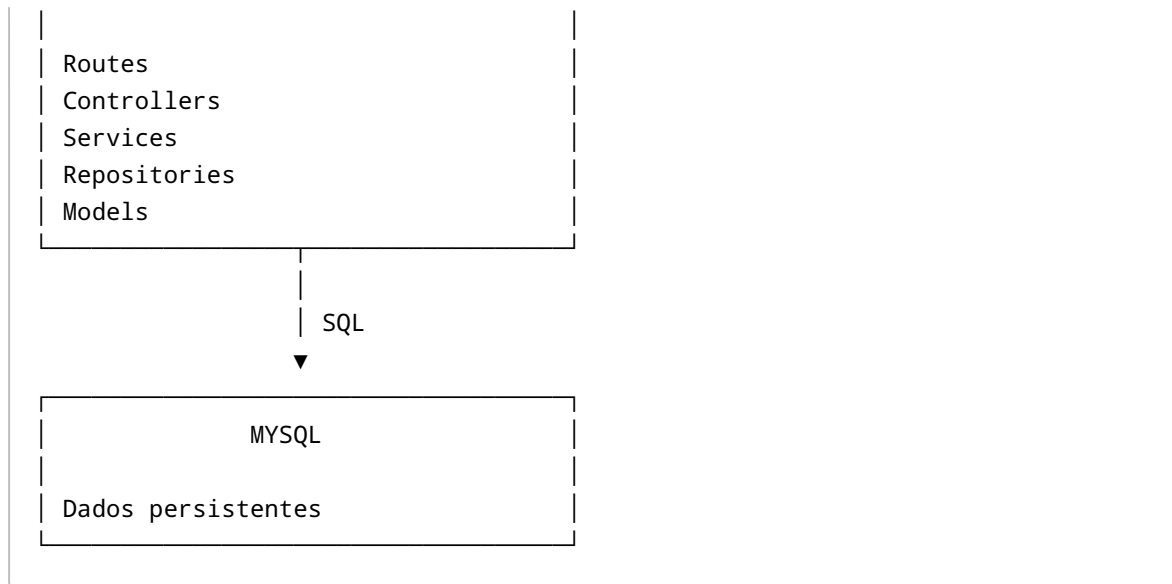
A ordem de serviço também deverá possuir relacionamento com:

MECÂNICO
USUÁRIO/FUNCIONÁRIO

20. ARQUITETURA PREVISTA

O sistema será dividido em três camadas principais:





21. TECNOLOGIAS

Front-end

- HTML5;
- CSS3;
- JavaScript.

Back-end

- Python;
- Flask;
- API REST.

Banco de dados

- MySQL.

Ferramentas

- Visual Studio Code;
 - WampServer;
 - MySQL;
 - Postman;
 - Git;
 - GitHub.
-

22. ORGANIZAÇÃO DOS REPOSITÓRIOS

O projeto será dividido em três repositórios principais.

APP_TORQUEHUB_BANCO

Responsável pela estrutura e scripts do banco.

```
APP_TORQUEHUB_BANCO/  
|  
├─ README.md  
├─ database.sql  
├─ inserts.sql  
├─ consultas.sql  
└─ diagramas/
```

APP_TORQUEHUB_API

Responsável pela API REST.

```
APP_TORQUEHUB_API/  
|  
├─ README.md  
├─ requirements.txt  
├─ app.py  
|  
├─ config/  
├─ routes/  
├─ controllers/  
├─ services/  
├─ repositories/  
├─ models/  
└─ utils/
```

APP_TORQUEHUB_INTERFACE

Responsável pela interface.

```
APP_TORQUEHUB_INTERFACE/  
|  
├─ README.md  
└─ index.html
```

```
|  
├─ pages/  
├─ components/  
├─ css/  
├─ js/  
├─ assets/  
└─ services/
```

23. API — RECURSOS PREVISTOS

A API deverá possuir recursos correspondentes às principais entidades.

Exemplos:

```
/api/auth  
  
/api/usuarios  
  
/api/clientes  
  
/api/veiculos  
  
/api/mecanicos  
  
/api/servicos  
  
/api/pecas  
  
/api/estoque  
  
/api/ordens-servico
```

As operações deverão utilizar os métodos HTTP adequados:

```
GET  
POST  
PUT/PATCH  
DELETE
```

O uso específico de cada verbo será definido durante a implementação da API.

24. INTERFACE DO SISTEMA

Tela 01 — Login

Deverá apresentar:

- Logo TorqueHub;
 - E-mail;
 - Senha;
 - Botão de acesso;
 - Mensagens de erro.
-

Tela 02 — Dashboard

Deverá apresentar:

- Total de clientes;
 - Total de veículos;
 - Ordens abertas;
 - Ordens em andamento;
 - Ordens finalizadas;
 - Estoque baixo;
 - Informações recentes.
-

Tela 03 — Clientes

Deverá permitir:

- Listar;
 - Pesquisar;
 - Cadastrar;
 - Editar;
 - Visualizar;
 - Desativar.
-

Tela 04 — Veículos

Deverá permitir:

- Listar;
- Pesquisar;
- Cadastrar;
- Editar;
- Visualizar;
- Consultar histórico.

Tela 05 — Mecânicos

Deverá permitir:

- Listar;
- Cadastrar;
- Editar;
- Visualizar;
- Desativar.

Tela 06 — Serviços

Deverá permitir:

- Listar;
- Cadastrar;
- Editar;
- Desativar.

Tela 07 — Estoque

Deverá permitir:

- Listar peças;
- Pesquisar;
- Cadastrar;
- Registrar entrada;
- Visualizar estoque;
- Identificar estoque baixo.

Tela 08 — Ordens de Serviço

Deverá permitir:

- Listar;
 - Pesquisar;
 - Filtrar;
 - Abrir ordem;
 - Visualizar;
 - Editar;
 - Atualizar status.
-

Tela 09 — Nova Ordem

Deverá permitir selecionar:

- Cliente;
 - Veículo;
 - Mecânico;
 - Problema relatado;
 - Quilometragem;
 - Previsão de entrega;
 - Serviços;
 - Peças;
 - Observações.
-

Tela 10 — Detalhes da Ordem

Deverá apresentar:

- Dados do cliente;
 - Dados do veículo;
 - Mecânico;
 - Problema;
 - Diagnóstico;
 - Serviços;
 - Peças;
 - Valor;
 - Status;
 - Observações;
 - Histórico da ordem.
-

Tela 11 — Histórico

Deverá permitir consultar o histórico de manutenção.

25. FUTURA EXPANSÃO — MÚLTIPLOS VEÍCULOS

A arquitetura do TorqueHub deverá permitir a evolução do modelo.

Versão 1.0

Veículo

└─ Motocicleta

Versão futura

```
Veículo
├─ Motocicleta
├─ Carro
├─ Caminhonete
├─ Van
└─ Utilitário
```

O sistema deverá utilizar atributos comuns sempre que possível.

Características específicas de determinado tipo de veículo poderão ser adicionadas posteriormente sem comprometer os dados comuns.

26. FUTURA EXPANSÃO — MÚLTIPLAS OFICINAS

Uma evolução planejada será permitir que o TorqueHub seja utilizado por várias oficinas.

A arquitetura futura poderá evoluir de:

```
TorqueHub
└─ Uma oficina
```

para:

```
TorqueHub
├─ Oficina A
├─ Oficina B
├─ Oficina C
└─ Oficina D
```

Cada oficina poderá possuir:

- Funcionários;
- Mecânicos;
- Serviços;
- Peças;
- Estoque;
- Clientes;
- Ordens de serviço.

27. FUTURA EXPANSÃO — CLIENTE FINAL

Em versões posteriores, o TorqueHub poderá possuir uma área específica para clientes.

O cliente poderá:

- Criar sua conta;
- Cadastrar veículos;
- Consultar histórico;
- Pesquisar oficinas;
- Favoritar oficinas;
- Solicitar orçamento;
- Agendar serviços;
- Acompanhar ordens;
- Avaliar oficinas.

28. FUTURA EXPANSÃO — OFICINAS FAVORITAS

O sistema poderá permitir que clientes favoritem oficinas.

A relação futura será:

Cliente N:N Oficina

Isso permitirá recursos como:

- Lista de oficinas favoritas;
- Quantidade de favoritos;
- Ranking;
- Preferências do cliente.

29. FUTURA EXPANSÃO — AVALIAÇÕES

Clientes poderão avaliar oficinas após a conclusão de serviços.

Uma avaliação poderá possuir:

- Nota;
- Comentário;
- Data;
- Cliente;
- Oficina;
- Ordem de serviço relacionada.

30. FUTURA EXPANSÃO — AGENDAMENTO

O sistema poderá futuramente permitir:

```
Cliente
↓
Oficina
↓
Serviço
↓
Data e horário
↓
Agendamento
```

31. FUTURA EXPANSÃO — MARKETPLACE

Em uma versão mais avançada, o TorqueHub poderá evoluir para uma plataforma de conexão entre proprietários de veículos e oficinas.

Fluxo planejado:

```
CLIENTE
↓
SELECIONA VEÍCULO
↓
ESCOLHE SERVIÇO
↓
PESQUISA OFICINAS
↓
SOLICITA ORÇAMENTO
↓
ESCOLHE OFICINA
↓
AGENDA
↓
REALIZA MANUTENÇÃO
↓
AVALIA OFICINA
```

32. CRITÉRIOS DE ACEITAÇÃO DA V1

A versão 1.0 será considerada funcional quando:

- O usuário conseguir realizar login;
- O sistema controlar permissões;
- Clientes puderem ser cadastrados;
- Veículos puderem ser cadastrados;
- Veículos puderem ser vinculados aos clientes;
- Mecânicos puderem ser cadastrados;
- Serviços puderem ser cadastrados;
- Peças puderem ser cadastradas;
- Estoque puder ser controlado;
- Ordens de serviço puderem ser abertas;
- Serviços puderem ser adicionados às ordens;
- Peças puderem ser adicionadas às ordens;
- O estoque for atualizado corretamente;
- O valor da ordem for calculado;
- O status da ordem puder ser atualizado;
- Ordens puderem ser finalizadas;
- Ordens puderem ser entregues;
- O histórico do veículo puder ser consultado;
- A API responder corretamente;
- O front-end consumir a API;
- Os dados forem persistidos no MySQL;
- O sistema puder ser executado localmente;
- Os endpoints puderem ser testados pelo Postman.

33. RESTRIÇÕES DO PROJETO

A primeira versão deverá respeitar as seguintes restrições:

1. O banco de dados deverá utilizar MySQL;
2. O back-end deverá utilizar Python e Flask;
3. O front-end deverá utilizar HTML, CSS e JavaScript;
4. A comunicação deverá ocorrer através de API REST;
5. A API deverá ser testada utilizando Postman;
6. O código deverá ser versionado com Git;
7. Os projetos deverão ser armazenados no GitHub;
8. A aplicação deverá funcionar em ambiente local;
9. A primeira versão terá foco em motocicletas;
10. A estrutura deverá permanecer preparada para novos tipos de veículos.

34. PREMISSAS

Considera-se que:

- A oficina possui acesso a computador;
 - Os funcionários possuem acesso ao sistema;
 - Os dados serão cadastrados pelos usuários autorizados;
 - A oficina possui conexão de rede para utilização do sistema web;
 - O ambiente de desenvolvimento utilizará WampServer para o MySQL;
 - O sistema inicialmente será utilizado em ambiente controlado.
-

35. EVOLUÇÃO PREVISTA

O projeto será desenvolvido seguindo uma estratégia incremental.

TorqueHub 1.0

Gestão interna de oficina

- Clientes;
- Veículos;
- Mecânicos;
- Serviços;
- Peças;
- Estoque;
- Ordens de serviço;
- Histórico;
- Dashboard.

TorqueHub 2.0

Suporte a múltiplos tipos de veículos

- Motocicletas;
- Carros;
- Caminhonetes;
- Vans;
- Outros.

TorqueHub 3.0

Gestão de múltiplas oficinas

- Empresas;
- Funcionários;
- Oficinas;
- Estoques independentes;
- Clientes;

- Serviços.

TorqueHub 4.0

Plataforma para clientes

- Conta do cliente;
- Veículos;
- Histórico;
- Oficinas;
- Favoritos;
- Avaliações.

TorqueHub 5.0

Marketplace automotivo

- Busca;
- Orçamentos;
- Agendamentos;
- Avaliações;
- Comparação de oficinas;
- Serviços;
- Notificações.

36. CONSIDERAÇÕES FINAIS

O TorqueHub será desenvolvido inicialmente como uma solução de gerenciamento para oficinas, com foco inicial em motocicletas.

Entretanto, o projeto não será modelado de forma limitada a esse tipo de veículo.

A utilização do conceito de **Veículo** como elemento central permitirá que a aplicação evolua futuramente para automóveis e outros veículos, preservando os principais módulos de clientes, ordens de serviço, serviços, peças, estoque e histórico.

A arquitetura também deverá permitir futuras expansões para múltiplas oficinas e, posteriormente, para uma plataforma de conexão entre clientes e estabelecimentos automotivos.

Dessa forma, a versão 1.0 terá um escopo controlado e adequado ao projeto acadêmico, enquanto sua arquitetura será planejada com visão de longo prazo.

O TorqueHub será desenvolvido seguindo uma separação clara entre interface, API e persistência de dados, permitindo evolução, manutenção e testes independentes das diferentes partes do sistema.

Fim do Documento de Requisitos — TorqueHub v1.0